

Incoming Request Processing Workflow

Product Requirement Document

Authored by Yosar Fatahillah

AI-assisted triage and remediation for customer operations teams

Table of Contents

- 1. Product Overview.....3
 - 1.1 The Problem.....3
 - 1.2 The Solution.....3
 - 1.3 Why This Approach Is Different.....3
 - 1.4 Target Users.....3
- 2. Objectives and Scope.....4
 - 2.1 Objectives.....4
 - 2.2 In Scope.....4
 - 2.3 Out of Scope.....4
- 3. Functional Requirements.....5
- 4. Classification and Decision Logic.....5
 - 4.1 Primary Engine: Claude API.....5
 - 4.2 Fallback Engine: Keyword Cascade.....6
 - 4.3 The Decision Tree.....6
- 5. Remediation Strategy per Branch.....6
- 6. Human Review Queue.....7
- 7. Stakeholder Directory & Notifications.....7
- 8. System Architecture.....7
 - 8.1 Data Flow.....7
 - 8.2 Repository Structure.....8
- 9. Technology Stack.....8
- 10. Non-Functional Requirements.....8
- 11. Setup and Running the Prototype.....9
- 12. Data Management.....9
- 13. Privacy and Responsible Use.....10
- 14. Known Limitations.....10
- 15. Future Roadmap.....10

1. Product Overview

1.1 The Problem

Operations teams in BPO and customer service environments receive a constant stream of incoming requests through email, web forms, and shared inboxes. Every one of these messages has to be read by a person, who then decides what kind of request it is, how urgent it is, and what should happen next. A billing dispute needs a very different response path from a general question, and an angry customer threatening to leave needs different handling again.

Doing this triage manually is slow, and the outcome depends heavily on who happens to pick the message up. Two agents can read the same complaint and route it two different ways. During busy periods, urgent cases can sit unnoticed in the same queue as routine questions. The cost shows up as inconsistent customer experience, missed escalations, and agent time spent on routing decisions rather than actually resolving issues.

1.2 The Solution

This prototype shows how AI can take over the triage step without taking the human out of the loop where it matters. For every incoming request, the system:

- classifies the request by type (Complaint, General Enquiry, Service Request, or Escalation) and urgency (Low, Medium, High, Critical) using the Anthropic Claude API,
- runs a distinct multi-step remediation workflow for that type, rather than sending one generic reply,
- generates the concrete outputs an operations team needs: draft customer responses, routing notifications, follow-up flags, and SLA timers,
- notifies the relevant stakeholder by real email through an admin-editable directory, with a simulated outbox fallback if email is not configured,
- holds escalations and low-confidence cases in a review queue where a supervisor approves or reclassifies them,
- records every decision and action in an SQLite audit trail that feeds a live dashboard, including an estimated time and cost saved calculator.

A key design feature is the confidence-gated escalation override. The classifier reports how confident it is in its own decision, and any low-confidence case is force-routed to human review instead of being auto-resolved. The system knows when it should not trust itself.

1.3 Why This Approach Is Different

Most AI triage demos stop at classification: they put a label on a message and call it done. This prototype treats classification as the first step of a workflow, not the end product. Each of the four request types triggers its own ordered sequence of downstream actions, and the outputs are written for an operations audience rather than as raw model responses. The prototype also keeps working when the AI does not: a deterministic keyword-based fallback engine takes over automatically if the API is unavailable, and a simulated outbox takes over if email sending fails, so the triage pipeline never simply goes down.

1.4 Target Users

- **Operations agents:** frontline agents, who receive pre-classified, pre-routed cases with a draft response already written.
- **Team leads:** supervisors and senior handlers, who get alerted to escalations by email and act on low-confidence cases from the review queue.
- **Operations managers:** managers, who use the dashboard and audit log to see request volumes, urgency mix, deflection rate, and estimated cost savings, and configure the stakeholder directory without a developer.

2. Objectives and Scope

2.1 Objectives

1. Accept an incoming request through a form, file upload, or simulated inbox.

2. Classify each request by type and urgency using an LLM.
3. Branch into a type-specific remediation workflow with at least two downstream steps per branch. This build implements four steps per branch.
4. Handle a minimum of three distinct request types. This build implements four.
5. Produce clearly legible outputs for an operations team: classification labels, action summaries, and generated artefacts such as draft responses and routing notifications.
6. Keep a human in control of critical and uncertain cases, with a working approve/reclassify mechanism, not just a flag.

2.2 In Scope

- Single-request processing through a Streamlit UI with three intake modes.
- Batch processing of the full sample dataset in one click, plus a CLI batch runner.
- A human review queue: approve a held case's handling, or reclassify it and re-run it down a different branch.
- An admin-editable stakeholder directory that controls notification routing, plus a notification outbox.
- Real email notifications to stakeholders via Gmail SMTP, with a simulated-outbox fallback if SMTP is not configured or fails.
- A summary dashboard showing volumes by type, urgency, and status, the deflection rate, and an editable estimated time/cost saved calculator.
- A persistent SQLite audit trail with a status filter and CSV export.
- A low-confidence escalation override as an edge-case safety mechanism.

2.3 Out of Scope

- Slack or ticketing-system integration (e.g. Zendesk). Email is the one live notification channel implemented; other channels are roadmap items.
- Multi-turn case threading. Each message is treated as a new request; repeat contacts from the same customer are not linked.
- User authentication and role-based access. The prototype is single-user.
- Production deployment concerns such as horizontal scaling, a networked database, and managed secrets storage.
- A labelled evaluation set measuring classification accuracy. The sample dataset demonstrates the pipeline but does not measure it.

3. Functional Requirements

ID	Requirement	Priority
FR-1	The system shall accept request text via a simulated inbox picker, manual text entry, or .txt file upload.	Must
FR-2	The system shall classify each request into one of four types with an urgency level, a sub-topic label, a confidence rating, and a one-line summary of key details.	Must
FR-3	The system shall route each classified request into a branch-specific remediation workflow of four ordered steps.	Must
FR-4	The system shall force-route any classification with Low confidence to the Escalation branch for human review, regardless of predicted type.	Must
FR-5	The system shall generate a draft customer-facing message appropriate to the branch (acknowledgement, answer, or confirmation).	Must
FR-6	The system shall log every processed case, including classification, engine used, steps executed, outputs, and lifecycle status, to a persistent audit trail.	Must
FR-7	The system shall notify the relevant stakeholder for Complaint, Service Request, and Escalation branches, by real email if configured or a logged simulated outbox otherwise.	Must

ID	Requirement	Priority
FR-8	The system shall provide a review queue where a human can approve a held case or reclassify it into a different branch, which re-runs that branch's remediation.	Must
FR-9	The system shall provide an admin-editable stakeholder directory (name, position, email) that controls notification routing without a code change.	Should
FR-10	The system shall provide a dashboard summarising volumes by type, urgency, and status, the deflection rate, and estimated agent time and cost saved with editable assumptions.	Should
FR-11	The system shall support batch processing of multiple requests in a single run.	Should
FR-12	The system shall fall back to a deterministic offline classifier when the Claude API is unavailable, and record which engine was used on every case.	Should
FR-13	The audit log shall be filterable by status and exportable as CSV.	Could

4. Classification and Decision Logic

4.1 Primary Engine: Claude API

The primary classification path sends the raw request text to Claude (model claude-sonnet-4-5) with a system prompt that defines each category in plain language, sets urgency criteria, and requires a strict JSON response. There is no scoring formula on this path. The model reads the message and reasons about it the way a trained agent would, then returns:

- request_type: Complaint, General Enquiry, Service Request, or Escalation
- urgency: Low, Medium, High, or Critical
- sub_topic: a short topic label such as billing dispute or password reset
- confidence: High, Medium, or Low, reflecting how sure the model is
- key_details: a one-sentence summary written for an operations agent

The prompt explicitly instructs the model that if it is torn between two categories, it should pick the more cautious one and report Low confidence. This makes the confidence field an honest signal rather than decoration, because the override mechanism depends on it.

4.2 Fallback Engine: Keyword Cascade

If no API key is configured, or an API call fails for any reason, classification falls back to a deterministic keyword engine. It scans the text against three regex pattern lists (escalation signals, complaint signals, and service signals), counts the hits in each, and applies a priority cascade: any escalation hit wins first, then complaint, then service, and anything with no hits at all defaults to General Enquiry.

Confidence on this path is derived from hit counts. Two or more hits in a category gives High confidence, a single hit gives Medium, and a message with no signals at all is treated as Medium confidence only if it looks like a genuine question (contains a question mark or runs to eight words or more). Short, vague, signal-free messages are rated Low confidence, which sends them to human review through the override. The engine field on every result records which path produced the classification, so nothing is hidden from the operations team.

4.3 The Decision Tree

The branching decision happens once per request, immediately after classification:

```
Request classified
|
v
Is confidence == Low?
|-- YES --> Force-route to Escalation branch (human review)
|-- NO --> Branch on request_type:
    Complaint    -> Complaint workflow
    General Enquiry -> Enquiry workflow
```

Service Request -> Service workflow
Escalation -> Escalation workflow

Once inside a branch, the remediation steps run strictly in sequence. The fork above is the only decision point; everything after it is a fixed, ordered pipeline for that branch.

5. Remediation Strategy per Branch

Each branch executes four ordered steps and produces a defined set of outputs. All steps within a branch run in the order listed.

Branch	Urgency	Steps (in order)	Outputs
Complaint	High	1. Acknowledge receipt 2. Escalate to senior handler (email) 3. Log case with priority flag 4. Set 2-hour follow-up reminder	Draft acknowledgement, escalation notification, case log entry with follow-up timestamp
General Enquiry	Low	1. Classify sub-topic 2. Generate response from knowledge base 3. Send response 4. Log as resolved	Auto-generated response, resolved status log
Service Request	Medium	1. Extract required details 2. Route to relevant department (email) 3. Generate confirmation to requester 4. Set 24-hour SLA timer	Routing notification, confirmation message, SLA flag
Escalation	Critical	1. Flag for human review 2. Draft urgent acknowledgement (held, not sent) 3. Notify supervisor (email) 4. Pause auto-resolution	Supervisor alert, held draft acknowledgement, human-in-the-loop flag

Three design choices are worth calling out. First, the Escalation branch deliberately drafts the acknowledgement but does not send it; a human should always review the message before it reaches an upset customer. Second, department routing in the Service Request branch uses keyword matching across five destinations (Billing, Technical, Account, Shipping, and a general Customer Care fallback); this is coarser than letting the model pick the department, but it is fast and fully explainable to an operations team. Third, stakeholder notifications for Complaint, Service Request, and Escalation are sent as real email when SMTP is configured, and fall back to a logged simulated outbox otherwise, so the workflow always completes.

6. Human Review Queue

Held cases (Escalations and low-confidence overrides) do not sit passively in the audit log. They surface in a dedicated review queue where a supervisor can:

- Approve the AI's handling, moving the case to an APPROVED status.
- Reclassify the case into a different branch, which re-runs that branch's full remediation sequence, including any stakeholder notification, against the corrected classification.

This closes the loop opened by the confidence override: the AI flags what it is unsure about, and a human makes the final call inside the same tool, rather than in a separate system.

7. Stakeholder Directory & Notifications

An Admin tab exposes an editable directory of stakeholders (name, position, email). Routing lookups match on a keyword within the position field (for example, 'Supervisor', 'Senior Handler', 'Department Owner - Billing'), so operations admins can change who gets notified for a given branch without a code change or redeploy.

Every notification, whether delivered as a real email or logged to the simulated outbox, is recorded with its delivery outcome, so the operations team can always audit who was told what and when.

8. System Architecture

8.1 Data Flow

Intake (Streamlit UI or CLI)

- > classifier.py (Claude API, offline fallback on failure)
- > workflows.py (confidence check, then branch handler)
 - > notifier.py (stakeholder email; simulated outbox on failure)
- > storage.py (SQLite: case log, status lifecycle, stakeholder directory)
- > Dashboard tab (pandas + Altair aggregation, ROI calculator)

8.2 Repository Structure

File	Role
app.py	Streamlit UI, five tabs: Process a Request, Batch Processing, Review Queue, Dashboard & Audit Log, Admin.
classifier.py	Claude API classification plus the offline keyword fallback. Returns type, urgency, sub-topic, confidence, key details, and the engine used.
workflows.py	The four branch handlers, the confidence-gated override, draft message generation, stakeholder lookup, and department routing.
storage.py	SQLite persistence: case log with status lifecycle, stakeholder directory, and dashboard summary aggregation.
notifier.py	Stakeholder email notifications via Gmail SMTP, with a simulated-outbox fallback.
sample_data.py	Ten synthetic sample requests covering all four branches plus a deliberately ambiguous edge case. AI-generated, no real customer data.
run_batch_demo.py	CLI batch runner that processes the full sample set and prints a readable trace plus a summary. Useful for smoke testing without the UI.
requirements.txt .env.example	Dependency pins and API key / SMTP setup template.

9. Technology Stack

Layer	Technology	Role
Language	Python 3	All orchestration and workflow logic
AI	Anthropic Claude API (claude-sonnet-4-5)	Request classification and customer-facing draft generation
UI	Streamlit	Five-tab prototype interface
Charting	Altair	Labelled bar charts on the dashboard
Storage	SQLite	Persistent case log, stakeholder directory, notification outbox
Email	smtplib (stdlib) + Gmail SMTP	Real stakeholder notifications, with a simulated-outbox fallback
Analysis	pandas	Dashboard aggregation and CSV export
Resilience	Regex keyword engine (built-in)	Deterministic fallback classification when the API is unavailable

10. Non-Functional Requirements

- **Resilience:** the pipeline must keep working when the LLM API or the email service is unavailable. Both have automatic fallbacks, and which path ran is recorded on every case and every notification.

- **Auditability:** every classification decision, override, step, output, and notification must be inspectable after the fact through the audit log and outbox.
- **Human oversight:** no critical or uncertain case may be auto-resolved. The Escalation branch, the confidence override, and the review queue's approve/reclassify controls all hold cases for a person.
- **Configurability:** notification routing is data, not code. Changing who gets notified requires editing the stakeholder directory, not the source.
- **Legibility:** outputs are written for operations staff, with plain labels, ordered step traces, and expandable detail rather than raw model output.
- **Responsiveness:** a single request processes in a few seconds with the live API, and near-instantly on the fallback engine.

11. Setup and Running the Prototype

```
pip install -r requirements.txt
export ANTHROPIC_API_KEY=sk-ant-... # optional, enables live AI
export SMTP_USER=you@gmail.com      # optional, enables live email
export SMTP_PASSWORD=your16charapppassword
streamlit run app.py                # full UI at localhost:8501
python3 run_batch_demo.py           # CLI smoke test, no UI needed
```

Without these keys the app runs entirely on the offline classifier and a simulated outbox, which keeps it demoable in restricted environments. The sidebar shows which mode is active for both AI and email.

12. Data Management

12.1 Schema (SQLite)

Table	Notes
cases	request_id, timestamp, raw_text, full classification result, engine, branch, override fields, steps_json, outputs_json, and a lifecycle status (RESOLVED, IN_PROGRESS, HELD_FOR_REVIEW, APPROVED).
stakeholders	name, position, email. Seeded with defaults on first run; fully replaceable from the Admin tab.
outbox	Every notification sent or simulated, with recipient, subject, body, and delivery outcome.

12.2 Sample Dataset

The prototype ships with ten synthetic requests generated with AI assistance, covering at least two examples per branch and one deliberately short, ambiguous message designed to trigger the low-confidence override. No proprietary or real customer data is used anywhere in the build.

13. Privacy and Responsible Use

- Request text is sent to the Anthropic API only when a key is configured. In a production setting this would need a data processing review, customer consent language, and redaction of personal identifiers before transmission.
- The audit trail stores raw request text locally in SQLite. Retention and access controls would need to be defined before real customer data touches the system.
- The system never auto-sends messages for critical cases. Draft-then-hold on the Escalation branch keeps a person accountable for the most sensitive communications.
- The engine field and the outbox's delivery column make AI and automation involvement transparent case by case, so an operations team can always tell whether a decision or a notification came from a live system or a fallback.

14. Known Limitations

Limitation	Detail and Recommended Action
No accuracy evaluation	The 10-message sample set demonstrates the pipeline but does not measure it. Build a labelled test set (~100 messages) with a per-category confusion matrix.
Urgency is displayed, not consumed	A High-urgency service request gets the same 24h SLA as a Medium one. Urgency should scale the SLA window, follow-up timing, and notification priority.
Keyword department routing	Fast and explainable, but coarser than letting the model choose the department directly. Keep the keyword rules as a fallback if upgraded.
No few-shot examples in the prompt	Category definitions only. Adding worked examples per category would tighten consistency on vague messages.
No case threading	Each message is a new case. A customer identifier would allow linking repeat contacts and auto-escalating on a pattern.
No feedback loop	Human reclassifications are logged but nothing yet learns from them, even though the data is already captured.
Single-user, local database	SQLite with no authentication. Needs a networked database and access control before any shared deployment.

15. Future Roadmap

- **Labelled evaluation set:** measure classification accuracy with a real confusion matrix, replacing demonstration with measurement.
- **Model-driven routing:** let the model choose the destination team directly, with the keyword rules retained as the fallback.
- **Urgency-scaled SLAs:** make urgency actually change SLA windows, follow-up timing, and notification priority, not just display a label.
- **Case threading:** link follow-up messages from the same customer to the existing case, enabling escalation on repeat contact.
- **Feedback loop:** use the already-logged human reclassifications to refine the classification prompt over time.
- **Additional channels:** Slack alerts and ticketing-system integration (e.g. Zendesk) alongside the existing email notifications.

This document accompanies the working prototype, the README, the Product Document, the Maintenance Document, the sample inputs and outputs, and the five-slide summary deck in the submission package.